
Colorfication

- Reconstructing Color from Grayscale -

Miniproject Report
Group 8

Aalborg University
The Technical Faculty of IT and Design



The Technical Faculty of IT and Design
Aalborg University
<http://www.aau.dk>

AALBORG UNIVERSITY

STUDENT REPORT

Title:

Colorfication

Theme:

Deep Learning

Project Period:

Fall Semester 2025

Project Group:

8

Participant(s):

Anne Sofie Ingerslev
Lærke Stein Raaschou
Rasmus Mølby Nielsen

Supervisor(s):

Andreas Aakerberg

Page Numbers: 22

Date of Completion:

November 21, 2025

The content of this report is freely available, but publication (with reference) may only be pursued due to agreement with the author.

Contents

1	Introduction	1
1.1	Usage of Generative Artificial Intelligence	1
2	Data	2
2.1	Places365	2
2.2	Data Loader	2
3	Methods	3
3.1	U-Net	3
3.1.1	Encoder	4
3.1.2	Decoder	4
3.1.3	Model Complexity	5
3.2	Generative Adversarial Network	5
3.2.1	Generator	5
3.2.2	Discriminator	6
3.2.3	PatchGAN for Colorization	6
3.2.4	Model Complexity	6
3.3	Summary	7
4	Experiments	8
4.1	U-Net	8
4.2	PatchGAN	14
5	Results	16
5.1	U-Net: Random and Pretrained Weights	16
5.2	PatchGAN	18
6	Conclusion	20
	Bibliography	21
A	Appendix	a

Chapter 1

Introduction

When a grayscale image is viewed, there is an intuitive sense of which colors are expected to appear in the corresponding color version. This miniproject aims to achieve that same intuition in an algorithm. The problem statement is formulated as follows:

"When given a grayscale image as input, how much of the original colors can be replicated using U-Net and PatchGAN algorithms?"

The goal of this project is to explore how well the different models and architectures can colorize grayscale images. Their performance is compared and evaluated using both qualitative and quantitative measures.

1.1 Usage of Generative Artificial Intelligence

This project uses generative AI as a supportive tool for various different parts of the project process. These include debugging code and latex document, grammatical support, finding relevant sources, and general consultation. All decisions, analyses, and implementations are under the responsibility of the authors.

Chapter 2

Data

This miniproject uses the dataset Places365, which is presented in this chapter.

2.1 Places365

The dataset used in this project is the MIT's Places365 dataset [1]. It is loaded using `torchvision.dataset`, which has a build in down-sampled version with 256x256 pixel images [2]. The dataset is split into a train, validation, and a test set with 365 different categories. The 'train-standard' split is used for training and contains around 1.8 million images. With this large amount of data, it is necessary to optimize the training process to avoid spending many hours training the models. This will be presented in Chapter 4.

2.2 Data Loader

The data loader is primarily used for loading the dataset and making batches that is iterable for the training. In this case, the data loader saves two versions of the image: One grayscale (input tensor) and one RGB (output tensor). In addition, data augmentation is used before transforming the images to tensors. This is not strictly necessary, due to the large amount of data, but it is done to gain experience in setting up a specialized data loader for future projects. The data augmentation includes: Vertical and horizontal flipping with a possibility of 0.25, and color jittering the brightness channel. The augmentation will increase the variation in the dataset.

Chapter 3

Methods

This chapter introduces the two different model approaches in this project: U-Net and PatchGAN. For each model, the relevant architecture and design choices are described. Both models are trained on the dataset presented in Chapter 2, to ensure a fair comparison.

3.1 U-Net

The main architecture used is U-Net. It was originally used for biomedical image segmentation, but has since been used for many different image-to-image tasks [3, 4]. The U-Net consists of two parts, an encoder to down-sample the image, and a decoder with skip-connections in between, to upscale the image, as illustrated in Figure 3.1. U-Net is useful for colorization because the encoder and decoder allow the model to capture global context while preserving fine details through skip connections.

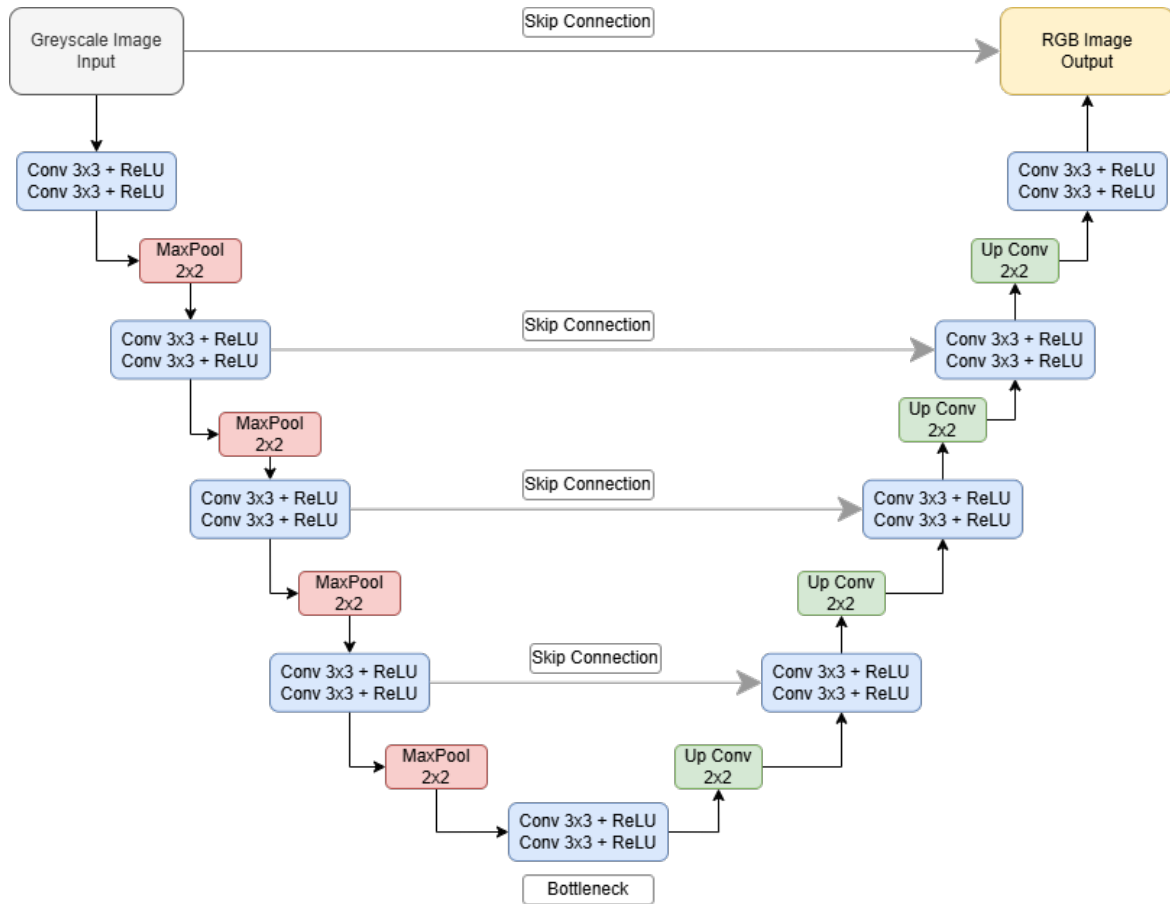


Figure 3.1: An illustration of the U-Net pipeline, inspired by the original paper [3].

The encoder and decoder described below follow the architectural choices for this project.

3.1.1 Encoder

The encoder is made up of 5 blocks that each contain two 3x3 convolutional layers with a padding of 1, each followed by ReLU activation. Pooling is in between the first four blocks, while the last block forms the bottleneck. The feature map depth increases from 1 up to 1024, while the spatial resolution is halved four times with a 2x2 MaxPooling operation with a stride of 2. This progressive down-sampling allows the model to capture global context.

3.1.2 Decoder

The decoder is made up of 4 up-sampling blocks, with each block beginning with a 2x2 up-convolution that doubles the spatial resolution, followed by concatenation with corresponding feature map from the encoder via skip connections. Afterwards, each block

applies two 3x3 convolutional layers with a padding of 1, each followed by ReLU activation to refine the combined features. This up-sampling allows the model to reconstruct a colored image while restoring the spatial details provided by the skip connections.

3.1.3 Model Complexity

The U-Net model takes a 256x256 grayscale image as input. In total, the model contains 31,030,723 learnable parameters. The computational demands for this model is calculated with Floating Point Operations per Second (FLOPs), as 109.09G total FLOPs.

3.2 Generative Adversarial Network

The Generative Adversarial Network (GAN) architecture consists of two main models, a generator and a discriminator, that are trained simultaneously, and work together to produce realistic images, as illustrated in Figure 3.2 below [5]. Several different types of GAN architecture exist, but this project uses U-Net for the generator and PatchGAN for the discriminator.

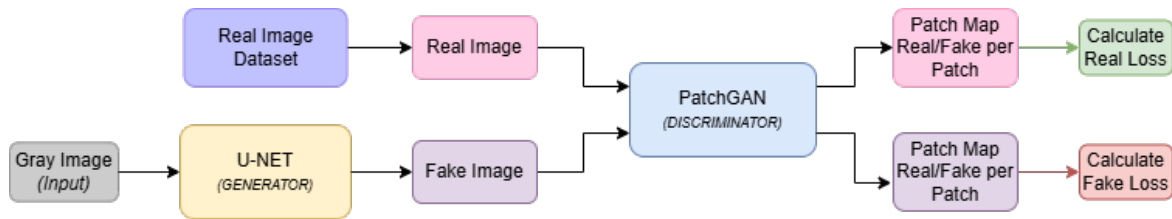


Figure 3.2: An illustration of the GAN pipeline used for colorization.

3.2.1 Generator

The purpose of the generator model is to create fake images that resemble real images. Its goal is to produce an output that is good enough to fool the discriminator. During training, the generator adapts its parameters, based on feedback from the discriminator model, by minimizing the generator loss function. As mentioned in Section 3.2, this project uses a U-Net architecture as the generating model. It has been chosen as the generator due to its ability to capture global context while preserving fine details as described in Section 3.1. This is an important quality in a colorization task, since accurate color prediction requires understanding the global context of the image while also preserving fine details for local objects [6, 3]. The pipeline is illustrated in Figure 3.3 below:

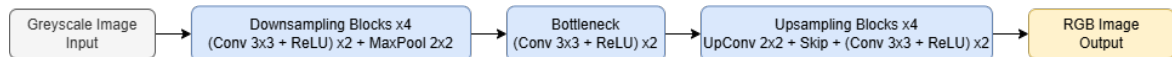


Figure 3.3: This pipeline illustrates how the generator works.

3.2.2 Discriminator

The discriminator model works as a binary classifier, to classify real images and fake images made by the generator model. Its ability to accurately distinguish between the two gradually improves through training, by adapting its parameters to minimize the discriminator loss function through backpropagation [7]. As mentioned in Section 3.2, PatchGAN has been chosen as the discriminator.

3.2.3 PatchGAN for Colorization

The PatchGAN architecture separates itself from other GAN discriminators by classifying local patches of an image, instead of the entire image [8, 7]. The architecture works by producing a map of values that correspond to the discriminator's real-fake score for each $N \times N$ patch in an image. However, it is important to note that the discriminator does not cut the image into patches. The final discriminator loss is calculated by averaging all patch values. The patch size is not manually chosen, but rather determined by the number of convolutional layers, kernel size, and stride. The original paper found that a patch size of 70×70 is the best performing, but a patch size of 16×16 produced almost similar results [8].

The discriminator in this project has 5 convolutional layers, each with a kernel size of 4, a stride of 2 in the first three layers, and a stride of 1 in the last two layers. This results in a patch size of 70×70 , which is identical to the best performing patch size in the original paper. Furthermore, the discriminator takes 4 channels as input, 1 for grayscale and 3 for RGB, because the colorization task requires pairs of grayscale and colored images. This pipeline is illustrated in Figure 3.4 below:

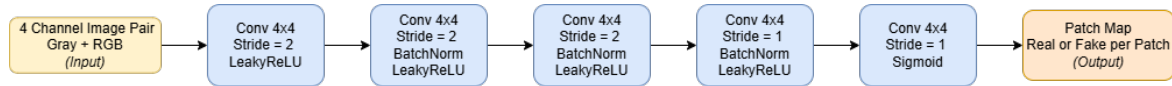


Figure 3.4: This pipeline illustrates how the discriminator works.

3.2.4 Model Complexity

The PatchGAN model takes a 256×256 image in 4 channels, 1 for grayscale and 3 for RGB, and a 256×256 image from the U-Net generator as input. The model complexity for the generator is described in Section 3.1.3, so only the discriminators model complexity is presented here. The discriminator contains 2.767.553 learnable parameters, with the computational demands as 50.68G total FLOPs.

3.3 Summary

In summary, this chapter presents the two models used in this project. U-Net is used as the main architecture because of its ability to capture global context while preserving fine details, which is useful for colorization tasks. The U-Net architecture is also combined with a PatchGAN discriminator to form a GAN-based approach, since the discriminator classifies local patches and thus is able to guide the generator towards producing realistic local colors.

Chapter 4

Experiments

The goal of this project is to investigate different models ability to bring back the color in grayscale images. Previously, two different models are presented: U-Net and PatchGAN. These models are the foundation for the experiments presented in this chapter.

The experiments consist of comparing the two models on their training, validation, and test. Performance is evaluated with MSE.

They are compared on the following parameters:

- Model complexity
- MAE score

4.1 U-Net

The U-Net architecture is trained from random weights. The first training set-up showed that running through over a million 256x256 images takes longer than 12 hours, therefore an updated training set-up is necessary. To optimize the training, multiple initial steps are done, including optimizing the GPU utilization and memory, saving model checkpoints, and using multiple GPU's. This decreased the training time to 3 epochs in 12 hours, which is still considered too long and inefficient.

To further optimize training, a 128x128 subpart of the image is randomly cropped out, decreasing the number of pixels with a factor of 4. This can have an effect on the model, because of the restricted receptive field, giving the model less context for the color prediction. For this test, the batch size is 256 uncropped images and 512 cropped images, to maximize the utilization of the GPU's. Due to the larger batch size, the training on cropped images requires fewer steps per epoch. This results in the two Figures 4.1 and

4.2 below somewhat uneven for comparison. However, after training for 3 epochs, the test shows very similar training loss for the two approaches, which can be seen in Figure 4.1.



Figure 4.1: Train loss for the models trained on uncropped and cropped images.

When comparing the validation MAE score for the two different approaches after 3 epochs, the difference is almost unnoticeable. The model trained on the uncropped images has a slightly better validation score of 0.058257, marked with a red circle in Figure 4.2, compared to the model trained on cropped images with a MAE of 0.061258, marked with a blue circle.

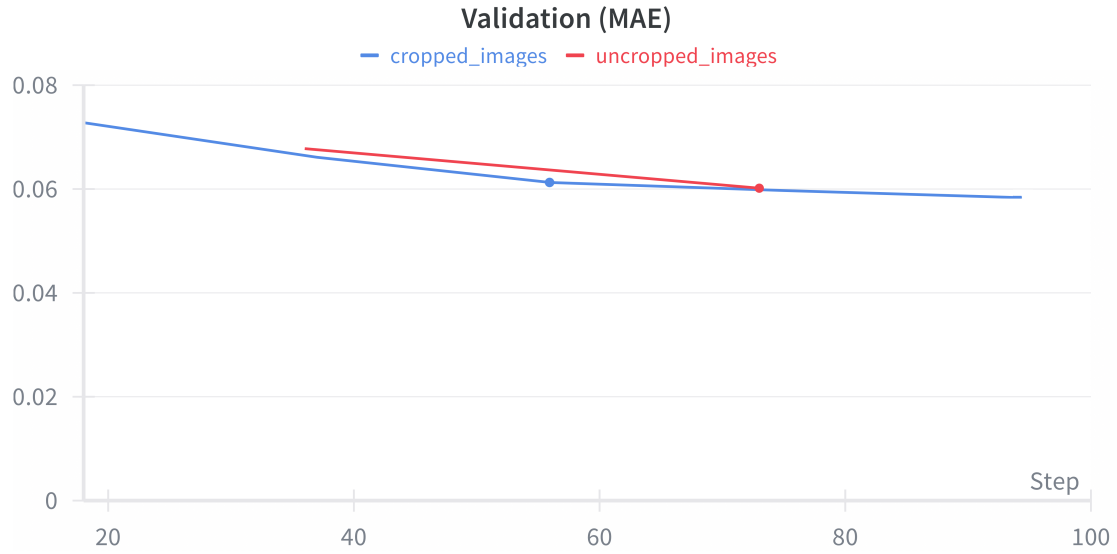


Figure 4.2: Plot of validation score for models trained on uncropped and cropped images.

The slight increase in L1 loss, for the model with the cropped images, can be caused by the fact that the model is trained on a quarter of the pixels the other model is trained on. The number of FLOPS also decreases to 27.27G total flops per images, decreasing this one fourth as well. This will give a faster training, making it possible to train more epochs, which will probably improve performance.

As the results from the early validation show that training on a cropped subpart of the image only increases the L1 loss slightly, random cropping will be used going forward.

The new cropped image model is trained on 4 GPU's with a batch size of 512. The model uses a SGD optimizer with momentum of 0.9, weight decay of 0.0001, and a learning rate of 1e-3. The training results presented in Figure 4.3 show that the training loss improves drastically at first before flattening out, with only slight improvements afterwards.



Figure 4.3: Plot of training loss, MSE for U-Net trained on cropped images.

The validation results for the model, presented in Figure 4.4, look similar to the training results. The validation score only improve slightly in the last epochs. It is possible that continuing the training could improve the results further, however, the improvement is unlikely to be significant. A changed training approach, such as a higher learning rate, could also be a suggestion for improving the model's capability to color images.

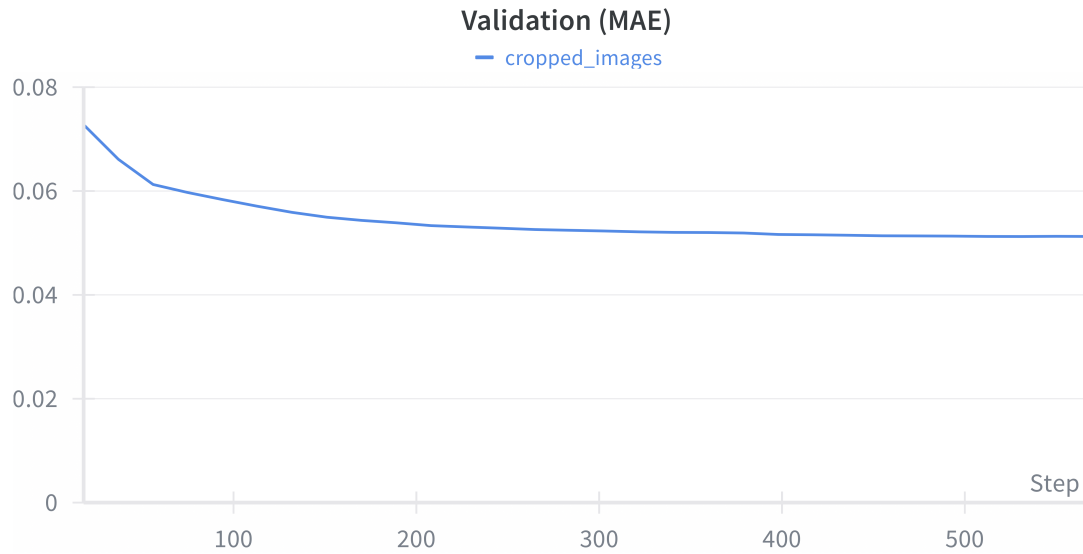


Figure 4.4: Plot of validation score, MAE for U-Net trained on cropped images.

As the model is trained from random weights, a long training time is expected to learn image composition. As an initial inspection of the predicted colorized output images from the validation show little to no coloring, a U-Net with pretrained weights is trained as well. The pretrained weights are trained for a segmentation task, but by changing the input and output layer of the U-Net, it is suitable for the colorization task [9]. The input layer is changed from three channels to one and the output layer is changed from 1 channel to three channels. It does not have ReLU as activation function in the output layer.

The pretrained U-Net shows a rapid drop in training loss in the beginning, with minimum change afterward. This is illustrated in Figure 4.5 below.

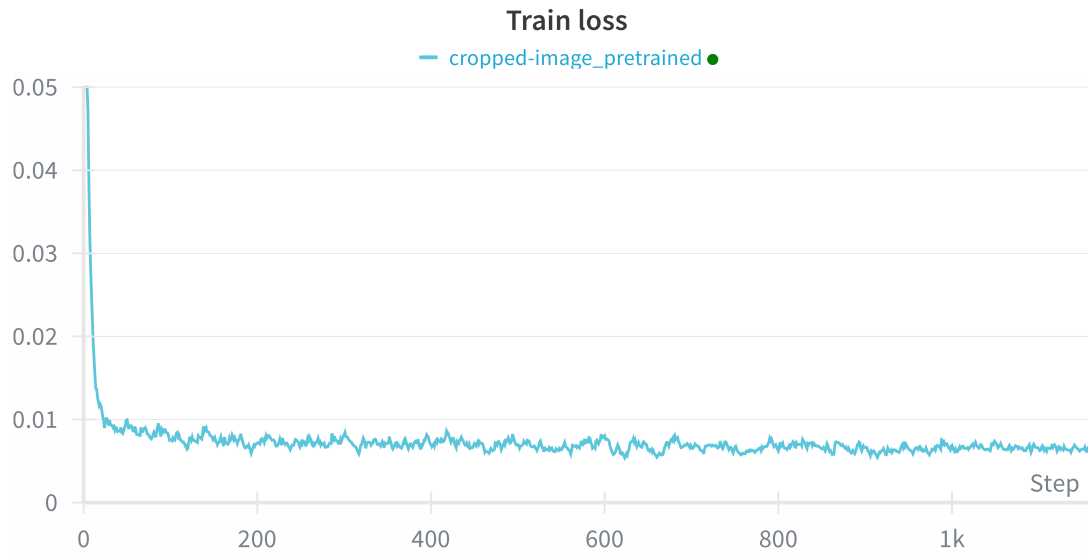


Figure 4.5: Train loss for pretrained U-Net.

The validation score for the pretrained U-Net is 0.050867, as shown in Figure 4.6.

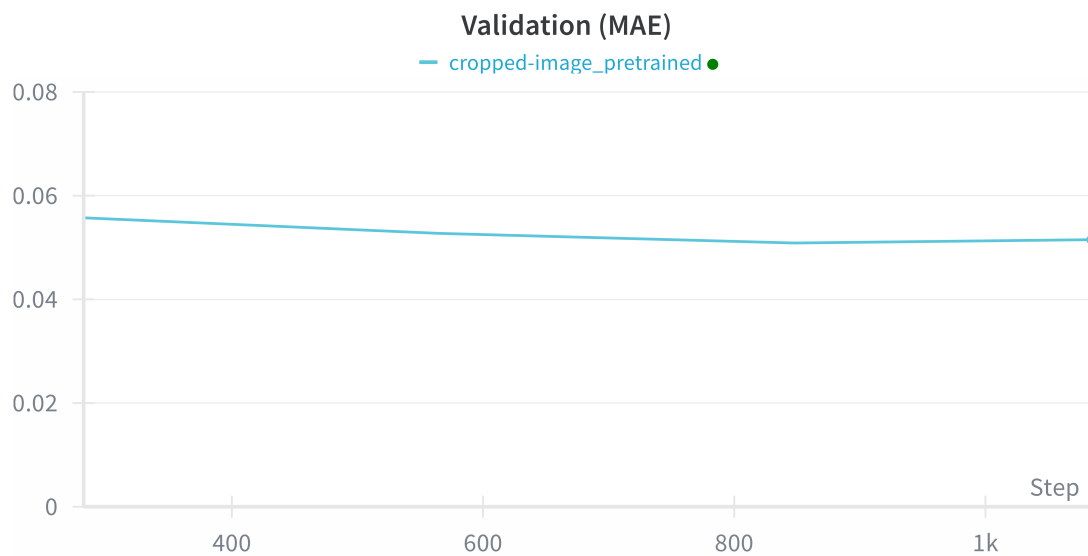


Figure 4.6: Plot of MAE validation score for pretrained U-Net.

4.2 PatchGAN

After experimenting with U-Net as a standalone model for colorization, as stated in Section 4.1, it was further setup with a PatchGAN discriminator for a GAN-based approach. The overall structure of U-Net and the Dataloader was not changed, the training script was only edited to be able to support the PatchGAN implementation. The architecture and parameters were inspired by following a Pix2Pix style setup, as it uses U-Net as a generator and PatchGAN as the discriminator [8]. The channel sizes are set to increase with every layer ending in 512 to give the model more capacity to learn important patterns.

To optimize the training and reduce computational costs, a 128x128 subpart of the image is randomly cropped out as well, decreasing FLOPs to 12.15G. The model is trained for 4 epochs, with a batch size of 254 on 4 GPU's, an Adam optimizer, and a learning rate of 0.0001. The adversarial loss for the generator swings significantly during training, as shown in Figure 4.7. This behavior is typical for GAN training, since the generator and discriminator are continuously competing. The instability does not necessarily indicate poor training performance.

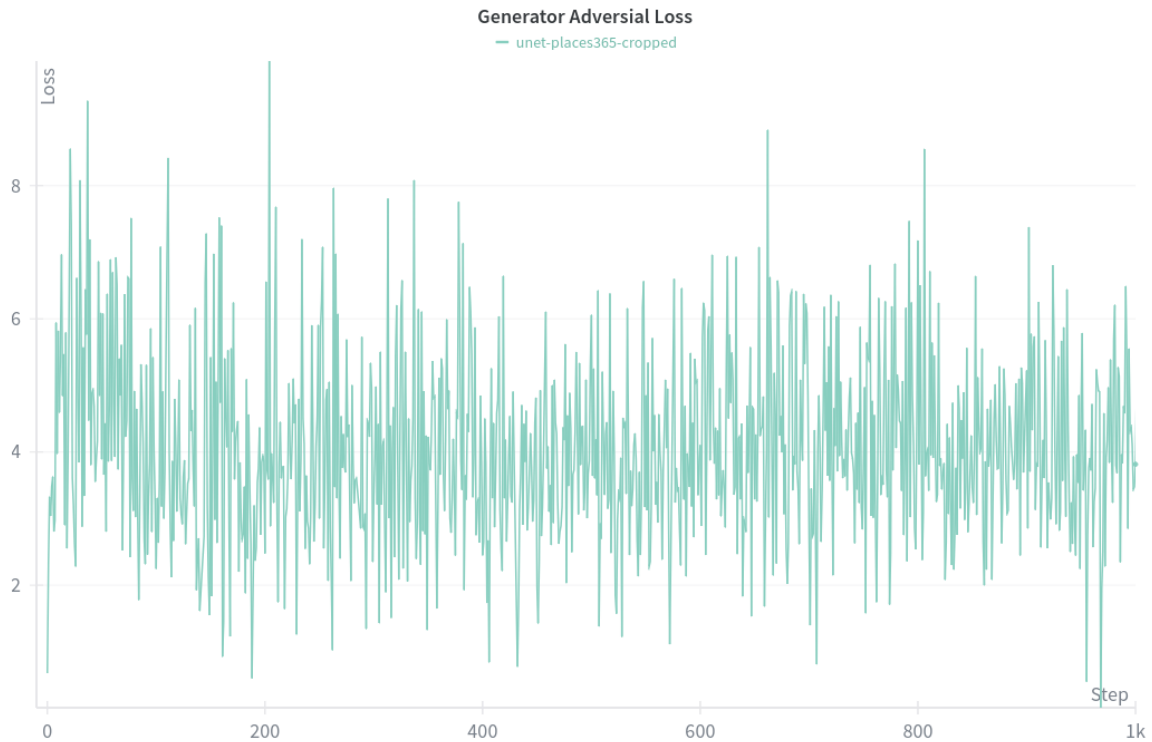


Figure 4.7: A graph showing the changing of the loss adversarial loss function of the generator using Binary Cross Entropy.

In contrast to the adversarial loss, the weighted L1 loss, presented in Figure 4.8), decreases rapidly at the beginning and then stabilizes. This indicates that the generator quickly learns the coarse structure of the target image, while fine details are refined more slowly.

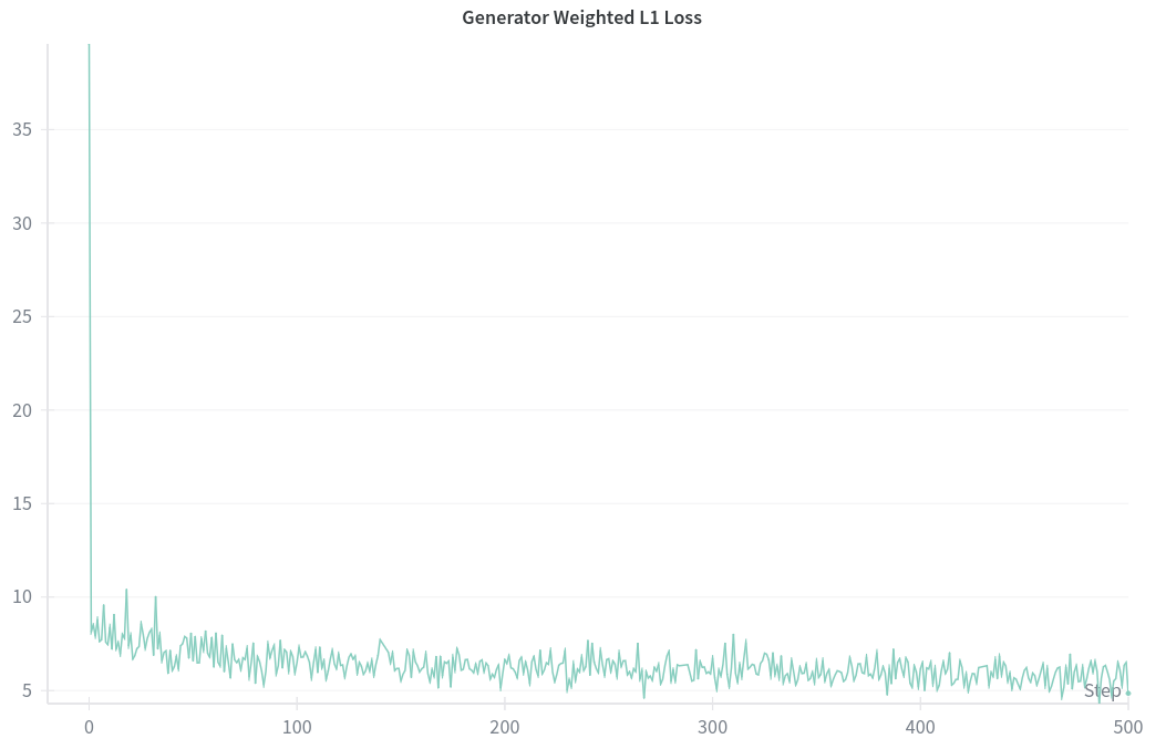


Figure 4.8: Weighted L1 loss for the generator throughout training.

Chapter 5

Results

The results from U-Net, pretrained U-Net, and PatchGAN are presented in this chapter.

5.1 U-Net: Random and Pretrained Weights

U-Net is tested on the test-set containing more than 300.000 images. The cropped image model is pulled after epoch 30 with a validation score of: 0.05125 MAE. Compared to the MAE from the test, presented in Table 5.1, the difference is 0.000183 which suggest that the model is not overfitted. However, the two different training approaches, random and pretrained weights, achieve almost identical MAE, with a difference of 0.000372. This could indicate that the model is underfitting.

Metric: MAE	Value
U-Net	0.051433
Pretrained U-Net	0.051061

Table 5.1: The results from U-Net on the test set.

An example to illustrate how well the U-Nets predict a colored image is presented for intuitive comparison in Figure 5.1.

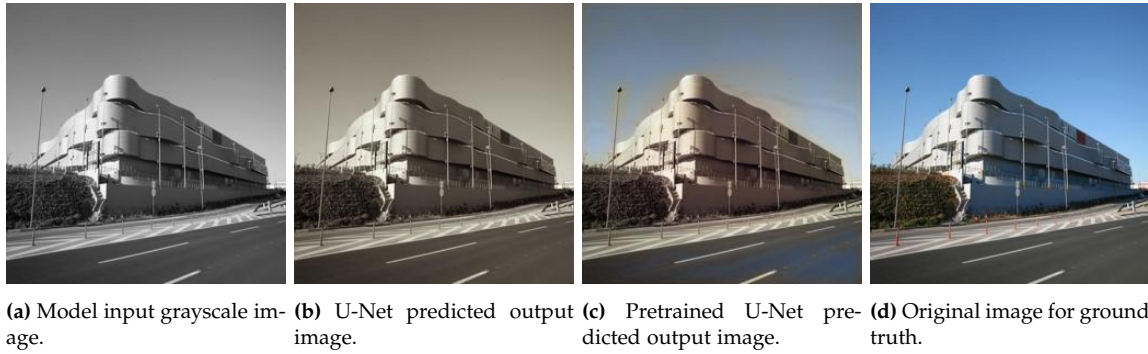


Figure 5.1: Comparison of grayscale input, U-Net prediction, pretrained U-Net prediction, and ground truth.

Based on the results presented in Figure 5.1 and Table 5.1, it is evident that U-Net does not perform well in this colorization task. The model works in an RGB colorspace, contrary to other papers that utilize LAB colorspace for this task [10]. Despite this, the paper "*Influence of Color Spaces for Deep Learning Image Colorization*" concludes that colorspace does not consistently affect model performance, so it cannot be concluded that the choice of RGB colorspace is a reason for U-Nets lack in performance [11]. However, three other factors have been identified as possible contributors to the lack in performance.

First, the model is trained with MSE/L2 loss, which means that it computes the square difference between each element in the predicted output and the corresponding element in the ground truth, and then takes the average across all elements [12]. This loss is useful when working with continuous values, but is more sensitive to outliers than MAE/L1 loss. Using MSE loss for training in a colorization task produces dull colors, as it encourages the model to predict the mean value for each plausible color, when multiple different options for the color of an object exist [13]. This may be a contributing factor to the low MSE performance.

Second, the activation function used in the output layer is ReLU where it should have been Sigmoid. This is because ReLU outputs values in the range $[0 - \infty]$, which is an unbounded positive output that allows color predictions to exceed the valid color range [14]. This can cause many issues, such as unstable training and color artifacts.

Third, the model seems to be underfitting, having difficulties adding any color to the image. This is a result of too little training, even though only slight improvement was observed when the model was pulled. This underfitting could as well be caused by setting non-optimal hyperparameters for this training. In future work, a higher starting learning-rate combined with a scheduler, or another optimizer, could potentially improve on the underfitting.

These three factors help explain why U-Net does not predict well.

However, when comparing U-Net and pretrained U-Net, there is a noticeable difference. This model actually predicts the color blue and does it in correct segments for the sky. Even though it also has some wrong blue colorization on the road, it is an improvement. This pattern also show on other images in the test set, see Appendix A. It appears that despite the difference in MAE between the two models is low, the pretrained U-Net benefits from pretrained weights for segmentation. ReLU as the activation function in the output layer is also removed.

5.2 PatchGAN

The test of the PatchGAN show an increase in MAE compared to the two U-Nets, as presented in Table 5.2. However, when inspecting Figure 5.2, it appears that this increase is caused by color artifacts. These artifacts negatively affect the pixel-wise error, but does not reflect the overall quality of the image.

Metric: MAE	Value
PatchGAN	0.060338
U-Net	0.051433
Pretrained U-Net	0.051061

Table 5.2: The results from PatchGAN on the test set, compared to the U-Nets.

An example to illustrate how well PatchGAN predicts a colored image is also presented for intuitive comparison in Figure 5.2.

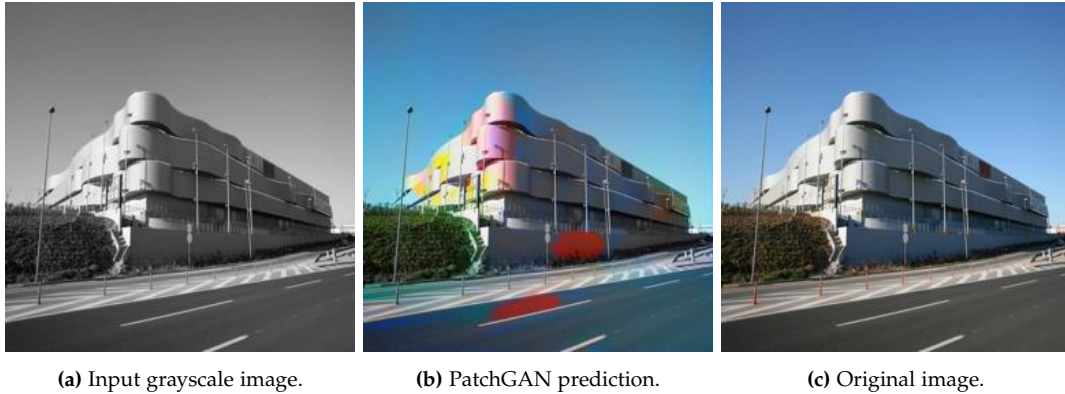


Figure 5.2: Comparison of grayscale input, PatchGAN prediction, and ground truth.

PatchGAN can color multiple colors compared to the U-Net with pretrained weights. It also has the abilities to color large segments of the image, although some color artifacts are introduced, like the red blob on the building, see Figure 5.2. This could be caused by

insufficient training and a too high learning rate. Additionally, the U-Net used as generator appears to be underfitted, which may also affect the performance, as the discriminator cannot teach meaningful corrections to the generator. In future work, an updated generator, longer training and a lower learning rate, could reduce the number of artifacts.

However, this result is overall better than the two U-Net models in visual comparison, despite the increased MAE. Moreover, while the U-Nets has 109.09G FLOPs, PatchGAN adds an additional 12.15G FLOPs, adding to the computational demands by approximately 11% when training the model. The additional FLOPs appear justified by the increase in performance.

Chapter 6

Conclusion

The objective of this project was to investigate the problem statement:

"When given a grayscale image as input, how much of the original colors can be replicated using U-Net and PatchGAN algorithms?"

In conclusion, the experiments show that this project's U-Net as a main architecture is not able to predict color in a grayscale image. This may be because of training with L2 loss, and having ReLU as the activation function in the output layer. Furthermore, adjusting the hyperparameters could be a way to improve the training, since the training in this project results in underfitting the model.

Experiments with the pretrained U-Net underlines this assumption, as a simple adjustment to the training, with pretrained weights and removing ReLU as activation function in the output layer, showed major visual improvements, despite the MAE difference being only 0.000372. The pretrained U-Net is capable of predicting the color blue and can do it in clear segments, but still fails overall. This model is also underfitted and could benefit from improved training.

Moreover, the experiments with PatchGAN show that this model outperforms both U-Nets in this project, despite having the highest MAE out of the three. It is able to produce multiple colors in correct segments, and generally predict accurately. However, the model also produced color artifacts, resulting in a higher MAE, which could be due to the generator being underfitted and therefore unable to utilize the discriminator. Improvements could include an updated generator, longer training and a lower learning rate.

In summary, PatchGAN predicted the most realistic colors, the pre-trained U-Net was able to predict blue, and the U-Net with random initialization did not succeed in predicting meaningful color.

Bibliography

- [1] Bolei Zhou et al. "Places: A 10 million Image Database for Scene Recognition". In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* (2017).
- [2] Pytorch. *Places365*. Accessed: 14-Nov-2025. URL: <https://docs.pytorch.org/vision/main/generated/torchvision.datasets.Places365.html#torchvision.datasets.Places365>.
- [3] Olaf Ronneberger, Philipp Fischer, and Thomas Brox. *U-Net: Convolutional Networks for Biomedical Image Segmentation*. Accessed: 14-Nov-2025. 2015. URL: <https://arxiv.org/pdf/1505.04597>.
- [4] Alejandro Ito Aramendia. *The U-Net : A Complete Guide*. Accessed: 14-Nov-2025. 2024. URL: <https://medium.com/@alejandritoaramendia/decoding-the-u-net-a-complete-guide-810b1c6d56d8>.
- [5] Rahul Roy. *Generative Adversarial Network (GAN)*. Accessed: 14-Nov-2025. 2025. URL: <https://www.geeksforgeeks.org/deep-learning/generative-adversarial-network-gan/>.
- [6] Richard Zhang, Phillip Isola, and Alexei A. Efros. *Colorful Image Colorization*. Accessed: 17-Nov-2025. 2016. URL: <https://arxiv.org/pdf/1603.08511>.
- [7] google for developers. *GAN Anatomy*. Accessed: 16-Nov-2025. 2025. URL: <https://developers.google.com/machine-learning/gan/training>.
- [8] Phillip Isola et al. *Image-to-Image Translation with Conditional Adversarial Networks*. Accessed: 17-Nov-2025. 2018. URL: <https://arxiv.org/pdf/1611.07004>.
- [9] milesial. *Pytorch U-Net*. Accessed: 20-Nov-2025. URL: <https://github.com/milesial/Pytorch-UNet>.
- [10] Justin Olah and Jenny Yang. *Let There Be Color: Deep Learning Image Colorization*. URL: <https://cs231n.stanford.edu/reports/2022/pdfs/109.pdf>.
- [11] Coloma Ballester et al. *Influence of Color Spaces for Deep Learning Image Colorization*. Accessed: 20-Nov-2025. 2022. URL: <https://arxiv.org/pdf/2204.02850>.
- [12] Bhipanshu Dhupar. *Loss Functions in Simple Autoencoders: MSE vs. L1 Loss*. Accessed: 20-Nov-2025. 2023. URL: <https://medium.com/@bhipanshudhupar/loss-functions-in-simple-autoencoders-mse-vs-l1-loss-4e838ae425b9>.

- [13] MINJE JWA and MYUNGJOO KANG. *GRAYSCALE IMAGE COLORIZATION USING A CONVOLUTIONAL NEURAL NETWORK*. Accessed: 20-Nov-2025. 2021. URL: <https://3dvar.com/Jwa2021Grayscale.pdf>.
- [14] geeksforgeeks. *ReLU Activation Function in Deep Learning*. Accessed: 20-Nov-2025. 2025. URL: <https://www.geeksforgeeks.org/deep-learning/relu-activation-function-in-deep-learning/>.

Appendix A

Appendix



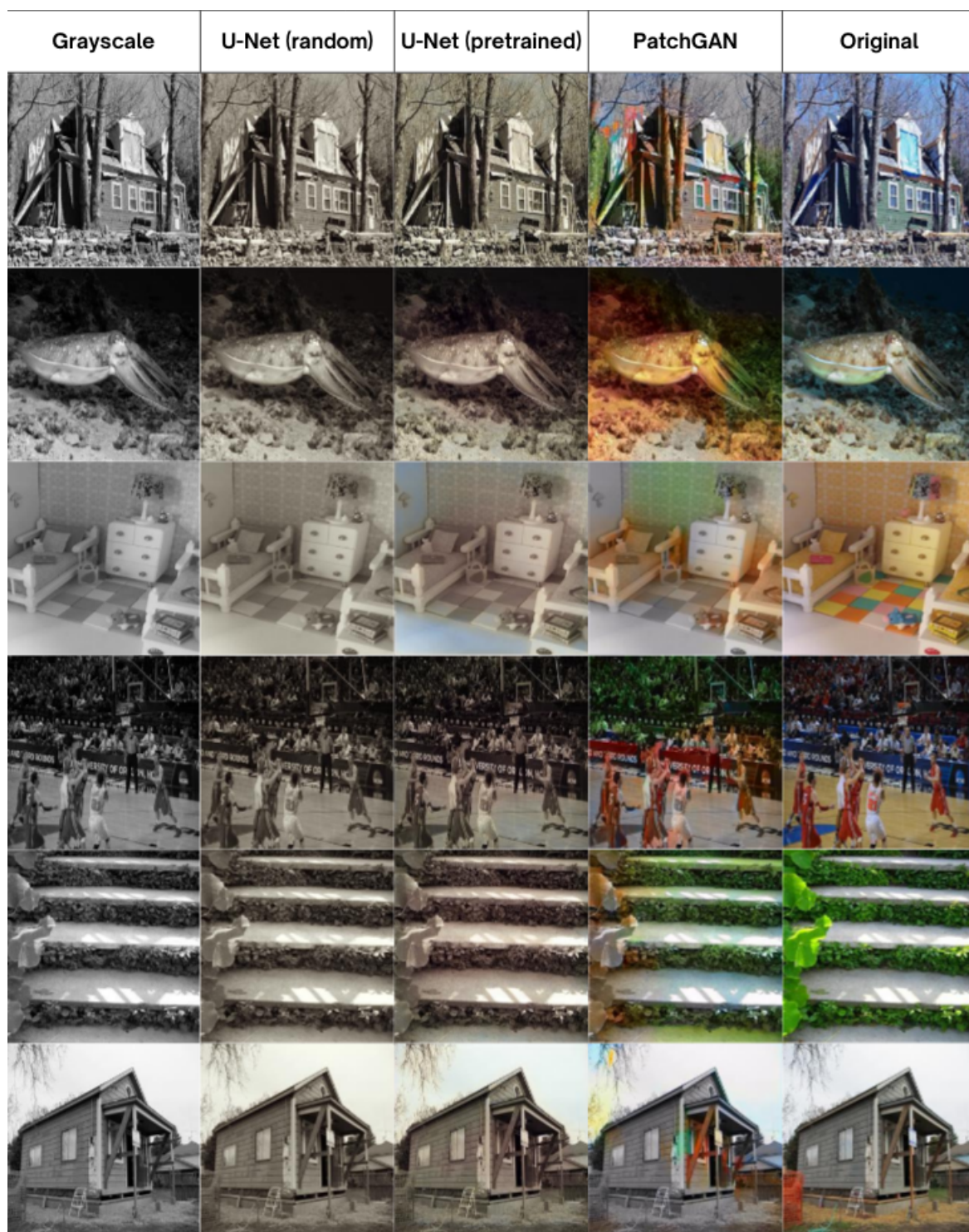


Figure A.2: Part 2 of test images from the three models.